Many-to-Many

This is an advanced relationship type that's implemented using a third table. For example, we could implement a third table called ContactMessage that holds both contact_id and sms_message_id columns. This third table acts as a join table between the associated schemas. This would be helpful to implement a feature like group chats, where the one-to-many relationship type doesn't work.

The many_to_many/3 function is used to define this relationship type. This function requires a third table, which can either be another schema or a database table name.

Each association can be customized with a variety of options. You can control the relationship field names (these default to association_name_id), customize the type (Ecto.UUID instead of integer IDs), and more. The documentation for each function lays out all of the options available to you.

Ecto's documentation has a cheatsheet[12] that shows all of the various association types, how to define them, and how to update them.

Let's finish this section by completing the other side of the contact relationship.

Write the Contact Schema

Create lib/phone_app/conversations/schema/contact.ex and add the following code to it:

phone_app/lib/phone_app/conversations/schema/contact.ex

```
defmodule PhoneApp.Conversations.Schema.Contact do
  use Ecto.Schema

  @timestamps_opts [type: :utc_datetime_usec]
  schema "contacts" do
    has_many :sms_messages, PhoneApp.Conversations.Schema.SmsMessage

    field :phone_number, :string
    field :name, :string

    timestamps()
  end
end
```

This looks similar to the SmsMessage schema, but we use the has_many/2 function to define the other side of the association.

Once you add this schema, your project will compile without warning about the broken association. We'll use this schema in the rest of this chapter. First, we need to create our database tables.

12. https://hexdocs.pm/ecto/associations.html